

ISTANBUL TECHNICAL UNIVERSITY
COMPUTER ENGINEERING DEPARTMENT

BLG 222E
COMPUTER ORGANIZATION
PROJECT 2 REPORT

CRN : 22596

LECTURER : Assoc. Prof. Dr. Gökhan İnce

GROUP MEMBERS:

150220937 : AMR NAEL ZUHDI TAWHEEL

150220116 : FURKAN YALÇIN

SPRING 2025

Contents

1	INTRODUCTION	1
1.1	Task Distribution	1
2	MATERIALS AND METHODS	1
2.1	2.1 Preliminary	1
2.2	2.2 Sequence Counter	2
2.3	2.3 Fetch	2
2.4	2.4 Decode (and First-Cycle Dispatch)	3
2.5	Instruction Execution	4
2.5.1	a-Instructions with Address References (Groups 1–5)	5
	Table of RTL micro-operations	5
	Group 1: Branch	5
	Group 2: Stack Operations	6
	Group 3: Subroutine	6
	Group 4: Immediate Loads	6
	Group 5: Direct and Offset Memory Operations	7
2.5.2	b-Instructions without Address Reference (Groups 6–10)	7
	Group 6: AR/DR Transfers	8
	Arithmetic and Logic Operations:	9
	Group 7: Increment and Decrement	9
	Group 8: Single-Operand Shifts and NOT	10
	Group 9: Two-Operand ALU Operations	10
	Group 10: Register-to-Register Move	11
3	RESULTS [15 points]	11
3.1	Results from Simulations We Built	11
3.2	Results from Instructor-Provided Simulations	18
3.3	Results from Simple Example	20
4	DISCUSSION	20
5	CONCLUSION	21

1 INTRODUCTION

In this project, we designed and implemented a complete CPU system that builds on the ALU subsystem developed in Project 1. Our CPU supports 36 instructions and has a RAM of 256 words (8 bits each). We began by carefully analyzing each instruction format to determine the precise micro-operations and control signals required in each clock cycle. The entire control logic were then described in Verilog HDL, and we verified the correctness of our implementation using the provided simulation files.

1.1 Task Distribution

Before starting this project, we divided the instructions between us, with each member responsible for implementing 18 instructions. During the implementation, we scheduled weekly meetings to follow the project's progress, and after each instruction was implemented, we reviewed it together to avoid any mistakes.

2 MATERIALS AND METHODS

2.1 Preliminary

Our primary task was to interpret the 36-instruction set and build a hardwired control unit around a microcycle counter. We reused all the modules from Project 1, then organized our CPU around the Fetch–Decode–Execute model, with a two-phase fetch, a common decode, and an instruction-specific execute phase. All registers (except `SP`, which is initialized to `8'hFF` at the bottom of memory) are cleared in an `initial` block. Control signals are then generated in dedicated `always @(posedge Clock)` blocks determined by bits of the sequence counter, ensuring each micro-operation occurs in its own clock cycle.

2.2 2.2 Sequence Counter

```
always @(posedge Clock) begin
  if(T_Reset == 1) begin
    T <= 12'h001;
    T_Reset <= 1'b0;
  end
  else begin
    if (T == 0)
      T <= 12'h001;
    else
      T <= T << 1;
    end
  end
end
```

Figure 1: 12-bit counter generating timing signals T[0]–T[11]

On reset (T_Reset=1), T is initialized to 12'h001, pointing at T[0], T_Reset is then cleared to allow normal counting. At each subsequent clock edge, T shifts left by one bit, so exactly one T[i] is high at any time (e.g. 00000001 → 00000010 → 00000100). When an instruction is completed, we set T_Reset to restart the cycle.

2.3 2.3 Fetch

Instruction fetch requires two microcycles:

- **T[0]:** Load IR(7:0) ← M[PC], PC ← PC + 1
- **T[1]:** Load IR(15:8) ← M[PC], PC ← PC + 1

By setting the memory chip-select and reading signals at T[0] and T[1], and selecting the LSB or MSB half of the IR via IR_L_H, we fetch the full 16-bit instruction over two cycles. We also disable all unrelated registers in these cycles to prevent accidental overwriting.

// on T[0]:

```
always @(posedge T[0]) begin
  disable_reg(),
  ARF_RegSel = 3'b100,    // enable PC as address source
  Mem_WR     = 1'b0,      // read
  Mem_CS     = 1'b0,      // chip select
  IR_L_H     = 1'b0,      // LSB half
  IR_Write   = 1'b1,      // load IR
  OutDSel    = 2'b00,     // PC -> address bus
  ARF_FunSel = 2'b01,     // PC <- PC + 1
end
```

```

end

// on T[1]:
always @(posedge T[1]) begin
    disable_reg(),
    ARF_RegSel = 3'b100,
    Mem_WR     = 1'b0,
    Mem_CS     = 1'b0,
    IR_L_H     = 1'b1,      // MSB half
    IR_Write   = 1'b1,
    OutDSel    = 2'b00,
    ARF_FunSel = 2'b01,
end

```

2.4 2.4 Decode (and First-Cycle Dispatch)

To minimize per-instruction latency, we perform both the decoding and the first execution step right at T[2]. By doing this, we avoid an extra cycle between fetch and execute.

```

always @(posedge T[2]) begin
    // Freeze all register enables so we can reconfigure control signals cleanly
    disable_reg(),

    // Capture the ALU's zero flag for conditional branches
    z      = ALUSys.ALU.FlagsOut[3],

    // Unpack the instruction word into our control variables
    Opcode  = ALUSys.IR.IROut[15:10],
    DestReg = ALUSys.IR.IROut[9:7],
    SrcReg1 = ALUSys.IR.IROut[6:4],
    RegSel  = ALUSys.IR.IROut[9:8],
    Address = ALUSys.IR.IROut[7:0],

    // Immediately drive the first set of control signals based on Opcode
    //   (e.g. for a branch, load the new PC, for an ALU op, issue the ALU function)
    case (Opcode)
        // ... first microcycle instructions
    end

```

```
endcase
```

```
T_Reset = (/* Reset the microcycle counter if the instruction is completed */),  
end
```

By decoding and dispatching in the same $T[2]$ edge, we optimized the total cycle count per instruction by one, accelerating overall program throughput.

2.5 Instruction Execution

All the RTLs we used to implement the address-reference instructions are shown in Tables 1 and 2. In all the RTLs below, we list every control signal needed in each microcycle. Since every transfer to or from memory is 16 or 32 bits whereas memory output bus is only 8 bits, each read or write takes multiple clock signals. In what follows, “first clock” always means the first execution microcycle at $T[2]$.

2.5.1 a-Instructions with Address References (Groups 1–5)

Table 1: Opcodes 0x00–0x08 (Groups 1–3)

Opcode	Symbol	RTL Micro-Operations	Grp
0x00	BRA	D0 T₂ : $PC \leftarrow \text{Value}(\text{IR}_{7:0}), SC \leftarrow 0.$	1
0x01	BNE	D1 T₂ : if $Z = 0$ then $PC \leftarrow \text{Value}(\text{IR}_{7:0}), SC \leftarrow 0,$ else $SC \leftarrow 0.$	1
0x02	BEQ	D2 T₂ : if $Z = 1$ then $PC \leftarrow \text{Value}(\text{IR}_{7:0}), SC \leftarrow 0$ else $SC \leftarrow 0.$	1
0x03	POPL	D3 T₂ : $SP \leftarrow SP + 1$ D3 T₃ : $DR[7:0] \leftarrow M[SP], SP \leftarrow SP + 1$ D3 T₄ : $DR[15:8] \leftarrow DR[7:0], DR[7:0] \leftarrow M[SP]$ D3 T₅ : $R_x[15:0] \leftarrow DR, SC \leftarrow 0.$	2
0x04	PSHL	D4 T₂ : $M[SP] \leftarrow R_x[7:0], SP \leftarrow SP - 1$ D4 T₃ : $M[SP] \leftarrow R_x[15:8], SP \leftarrow SP - 1, SC \leftarrow 0.$	2
0x05	POPH	D5 T₂ : $SP \leftarrow SP + 1$ D5 T₃ : $DR[7:0] \leftarrow M[SP], SP \leftarrow SP + 1$ D5 T₄ : $DR[15:8] \leftarrow DR[7:0], DR[7:0] \leftarrow M[SP], SP \leftarrow SP + 1$ D5 T₅ : $DR[23:7] \leftarrow DR[15:0], DR[7:0] \leftarrow M[SP], SP \leftarrow SP + 1$ D5 T₆ : $DR[31:7] \leftarrow DR[23:0], DR[7:0] \leftarrow M[SP]$ D5 T₇ : $R_x \leftarrow DR, SC \leftarrow 0.$	2
0x06	PSHH	D6 T₂ : $M[SP] \leftarrow R_x[7:0], SP \leftarrow SP - 1$ D6 T₃ : $M[SP] \leftarrow R_x[15:8], SP \leftarrow SP - 1$ D6 T₄ : $M[SP] \leftarrow R_x[23:16], SP \leftarrow SP - 1$ D6 T₅ : $M[SP] \leftarrow R_x[31:24], SP \leftarrow SP - 1, SC \leftarrow 0.$	2
0x07	CALL	D7 T₂ : $M[SP] \leftarrow PC[7:0], SP \leftarrow SP - 1$ D7 T₃ : $M[SP] \leftarrow PC[15:8], SP \leftarrow SP - 1$ D7 T₄ : $PC \leftarrow \text{Value}(\text{IR}_{7:0}), SC \leftarrow 0.$	3
0x08	RET	D8 T₂ : $SP \leftarrow SP + 1$ D8 T₃ : $DR[7:0] \leftarrow M[SP], SP \leftarrow SP + 1$ D8 T₄ : $DR[15:8] \leftarrow DR[7:0], DR[7:0] \leftarrow M[SP]$ D8 T₅ : $PC \leftarrow DR[15:0], SC \leftarrow 0.$	3

Table of RTL micro-operations

Group 1: Branch These three opcodes (BRA, BNE, BEQ) all compute a new PC by adding the eight-bit immediate ($\text{IR}_{7:0}$) to the old PC. BRA always takes it, BNE only if $Z = 0$, BEQ only if $Z = 1$. None save a return address, and each completes in 3 microcycles two fetching cycles and one execution cycle.

Group 2: Stack Operations POPL/POPH and PSHL/PSHH push or pop entire words to/from memory through SP. POPL and POPH variants increment SP and read the data into a data register (DR) before writing back to R_x , taking 3 (16-bit) or 5 (32-bit) cycles respectively. PSH and PSHH variants write R_x 's bytes to memory and decrease SP, taking 2 or 4 cycles respectively.

Group 3: Subroutine CALL pushes the two PC bytes onto the stack then loads $PC \leftarrow IR_{7:0}$ (3 cycles). RET pops two bytes into PC (4 cycles).

Table 2: Opcodes 0x19,0x1A,0x1E–0x20,0x24 (Groups 4–5)

Opcode	Symbol	RTL Micro-Operations	Grp
0x19	MOVL	D25 T₂: $R_x[7:0] \leftarrow IR_{7:0}$ D25 T₃: <i>pass through ALU to update flags</i> , $SC \leftarrow 0$	4
0x1A	MOVSH	D26 T₂: $R_x[31:8] \leftarrow R_x[23:0]$ $R_x[7:0] \leftarrow IR_{7:0}$ D26 T₃: <i>pass through ALU to update flags</i> , $SC \leftarrow 0$	4
0x1E	LDAL	D30 T₂: $AR \leftarrow IR_{7:0}$ D30 T₃: $DR[31:8] \leftarrow 0$, $DR[7:0] \leftarrow M[AR]$, $AR \leftarrow AR+1$ D30 T₄: $DR[15:8] \leftarrow DR[7:0]$, $DR[7:0] \leftarrow M[AR]$ D30 T₅: $R_x \leftarrow DR$, $SC \leftarrow 0$.	5
0x1F	LDAH	D31 T₂: $AR \leftarrow IR_{7:0}$ D31 T₃: $DR[31:8] \leftarrow 0$, $DR[7:0] \leftarrow M[AR]$, $AR \leftarrow AR+1$ D31 T₄: $DR[15:8] \leftarrow DR[7:0]$, $DR[7:0] \leftarrow M[AR]$, $AR \leftarrow AR+1$ D31 T₅: $DR[23:8] \leftarrow DR[15:0]$, $DR[7:0] \leftarrow M[AR]$, $AR \leftarrow AR+1$ D31 T₆: $DR[31:8] \leftarrow DR[23:0]$, $DR[7:0] \leftarrow M[AR]$ D31 T₇: $R_x \leftarrow DR$, $SC \leftarrow 0$.	5
0x20	STA	D32 T₂: $AR \leftarrow IR_{7:0}$ D32 T₃: $M[AR] \leftarrow R_x[31:24]$, $AR \leftarrow AR+1$ D32 T₄: $M[AR] \leftarrow R_x[23:16]$, $AR \leftarrow AR+1$ D32 T₅: $M[AR] \leftarrow R_x[15:8]$, $AR \leftarrow AR+1$ D32 T₆: $M[AR] \leftarrow R_x[7:0]$, $SC \leftarrow 0$.	5
0x24	STRIM	D36 T₂: $S_1 \leftarrow IR_{7:0}$ D36 T₃: $AR \leftarrow AR + \text{Value}(IR_{7:0})$ D36 T₄: $M[AR] \leftarrow R_x[31:24]$, $AR \leftarrow AR+1$ D36 T₅: $M[AR] \leftarrow R_x[23:16]$, $AR \leftarrow AR+1$ D36 T₆: $M[AR] \leftarrow R_x[15:8]$, $AR \leftarrow AR+1$ D36 T₇: $M[AR] \leftarrow R_x[7:0]$, $SC \leftarrow 0$.	5

Group 4: Immediate Loads This group contains the two immediate-to-register moves, MOVL (0x19) and MOVSH (0x1A) after the move operation they update the ALU flags. Both

of them requires two microcycles.

- **MOVL** zero-extends the 8-bit immediate ($IR_{7:0}$) into the low byte of R_x , clearing the upper bits.
- **MOVSH** first shifts R_x left by 8 bits, then injects $IR_{7:0}$ into $R_x[7:0]$.

Both instructions complete after the ALU flag update at $T[3]$.

Group 5: Direct and Offset Memory Operations These four instructions use $IR_{7:0}$ either as an address (**LDAL/LDAH/STA**) or as an offset (**STRIM**). All begin at $T[2]$:

- **LDAL** (0x1E) and **LDAH** (0x1F) load a 16-bit or 32-bit word from $M[IR_{7:0}]$ into DR over successive byte-reads, then write $DR \rightarrow R_x$ at the final cycle.
- **STA** (0x20) stores the 32-bit contents of R_x to $M[IR_{7:0}]$ by writing each byte in turn (T_3-T_6).
- **STRIM** (0x24) first adds the 8-bit offset $IR_{7:0}$ to AR (T_3), then writes the four bytes of R_x to $M[AR]$, $AR \leftarrow AR+1$ on each subsequent cycle (T_4-T_7).

All end by clearing SC at the final microcycle, ensuring the next fetch begins at $T[0]$.

2.5.2 b-Instructions without Address Reference (Groups 6–10)

All the RTLs for non-addressing instructions are shown in Tables 3, 4 and 5. In each case the first execution microcycle is at $T[2]$.

Table 3: Opcodes 0x1B–0x1D, 0x21–0x23 (AR/DR Transfers)

Opcode	Symbol	RTL Micro-Operations	Grp
0x1B	LDARL	D27 T₂ : $DR[7:0] \leftarrow M[AR]$, $AR \leftarrow AR+1$, $DR[31:8] \leftarrow 0$ D27 T₃ : $DR[15:8] \leftarrow DR[7:0]$, $DR[7:0] \leftarrow M[AR]$ D27 T₄ : $DSTREG \leftarrow DR$, $SC \leftarrow 0$.	6
0x1C	LDARH	D28 T₂ : $DR[7:0] \leftarrow M[AR]$, $AR \leftarrow AR+1$, $DR[31:8] \leftarrow 0$ D28 T₃ : $DR[15:8] \leftarrow DR[7:0]$ $DR[7:0] \leftarrow M[AR]$, $AR \leftarrow AR+1$ D28 T₄ : $DR[23:8] \leftarrow DR[15:0]$, $DR[7:0] \leftarrow M[AR]$, $AR \leftarrow AR+1$, D28 T₅ : $DR[31:8] \leftarrow DR[23:0]$, $DR[7:0] \leftarrow M[AR]$, D28 T₆ : $DSTREG \leftarrow DR$, $SC \leftarrow 0$.	6
0x1D	STAR	D29 T₂ : $M[AR] \leftarrow SREG1[31:24]$, $AR \leftarrow AR+1$ D29 T₃ : $M[AR] \leftarrow SREG1[23:16]$, $AR \leftarrow AR+1$ D29 T₄ : $M[AR] \leftarrow SREG1[15:8]$, $AR \leftarrow AR+1$ D29 T₅ : $M[AR] \leftarrow SREG1[7:0]$, $SC \leftarrow 0$.	6
0x21	LDDRL	D33 T₂ : $DR[31:8] \leftarrow 0$, $DR[7:0] \leftarrow M[AR]$, $AR \leftarrow AR+1$ D33 T₃ : $DR[15:8] \leftarrow DR[7:0]$, $DR[7:0] \leftarrow M[AR]$, $SC \leftarrow 0$.	6
0x22	LDDRH	D34 T₂ : $DR[31:8] \leftarrow 0$, $DR[7:0] \leftarrow M[AR]$, $AR \leftarrow AR+1$ D34 T₃ : $DR[15:8] \leftarrow DR[7:0]$, $DR[7:0] \leftarrow M[AR]$, $AR \leftarrow AR+1$ D34 T₄ : $DR[23:8] \leftarrow DR[15:0]$, $DR[7:0] \leftarrow M[AR]$, $AR \leftarrow AR+1$ D34 T₅ : $DR[31:8] \leftarrow DR[23:0]$, $DR[7:0] \leftarrow M[AR]$, $SC \leftarrow 0$.	6
0x23	STDR	D35 T₂ : $DSTREG \leftarrow DR$, $SC \leftarrow 0$.	6

Group 6: AR/DR Transfers This group implements the transfers between the Address Register (AR), the Data Register (DR), memory, and the destination register (DSTREG). Each of these instructions (except STDR) uses the AR to step through consecutive memory bytes and assembles or disassembles full words in DR before moving them to or from the register file (RF).

- **LDARL (0x1B)** reads a 16-bit word from $M[AR]$ into DR over two byte-reads ($D27T_2$ – $D27T_3$), first clearing $DR[31:8]$, then writing $DR[15:0]$ into the destination register at the end of $D27T_4$.
- **LDARH (0x1C)** reads a 32-bit word from $M[AR]$ into DR over four byte-reads

($D28T_2$ – $D28T_5$), clearing $DR[31:8]$ at the start and finally moving $DR[31:0]$ into $DSTREG$ at $D28T_6$.

- **STAR (0x1D)** stores the 32-bit contents of $SREG1$ to memory at AR , writing each byte from MSB to LSB over four cycles ($D29T_2$ – $D29T_5$), incrementing AR after each byte.
- **LDDRL (0x21)** loads a 16-bit word into DR , requires two cycles two read.
- **LDDRH (0x22)** loads a 32-bit word in the same way but over four byte-reads ($D34T_2$ – $D34T_5$), each time shifting the previously read byte into the next field of DR , then clears SC .
- **STDRL (0x23)** completes in a single microcycle ($D35T_2$) by moving the full 32-bit DR into the destination register and clearing SC .

Table 4: Opcodes 0x09–0x10 (Groups 7–8)

Opcode	Symbol	RTL Micro-Operations	Grp
0x09	INC	D9 T₂ : $DSTREG \leftarrow SREG1$ D9 T₃ : $DSTREG \leftarrow DSTREG + 1$ D9 T₄ : $DSTREG \leftarrow DSTREG, SC \leftarrow 0$.	7
0x0A	DEC	D10 T₂ : $DSTREG \leftarrow SREG1$ D10 T₃ : $DSTREG \leftarrow DSTREG - 1$ D10 T₄ : $DSTREG \leftarrow DSTREG, SC \leftarrow 0$.	7
0x0B	LSL	D11 T₂ : $DSTREG \leftarrow LSL(SREG1), SC \leftarrow 0$.	8
0x0C	LSR	D12 T₂ : $DSTREG \leftarrow LSR(SREG1), SC \leftarrow 0$.	8
0x0D	ASR	D13 T₂ : $DSTREG \leftarrow ASR(SREG1), SC \leftarrow 0$.	8
0x0E	CSL	D14 T₂ : $DSTREG \leftarrow CSL(SREG1), SC \leftarrow 0$.	8
0x0F	CSR	D15 T₂ : $DSTREG \leftarrow CSR(SREG1), SC \leftarrow 0$.	8
0x10	NOT	D16 T₂ : $DSTREG \leftarrow NOT(SREG1), SC \leftarrow 0$.	8

Arithmetic and Logic Operations: For these instructions, we directly used our ALU subsystem, which supports all the required bitwise and shift operations.

Group 7: Increment and Decrement Opcodes 0x09 (INC) and 0x0A (DEC) each take three microcycles:

- At $D9T_2/D10T_2$, the register file moves $SREG1$ into $DSTREG$ directly.

- At $D9T_3/D10T_3$, the register file applies its internal increment/decrement logic to DSTREG (without affecting the flags).
- At $D9T_4/D10T_4$, the updated DSTREG is routed through the ALU in a pass-through operation—this updates the condition flags—and then written back to DSTREG. Finally, SC is cleared to start the next instruction.

Group 8: Single-Operand Shifts and NOT Opcodes 0x0B–0x0F (LSL, LSR, ASR, CSL, CSR) and 0x10 (NOT) each complete in one microcycle ($D11T_2$ – $D16T_2$ respectively). At each cycle, the ALU:

- Retrieves SREG1 as its input,
- Performs the specified shift or bitwise NOT,
- Writes the result directly into DSTREG, and
- Clears SC so the next fetch can begin immediately.

Table 5: Opcodes 0x11–0x18 (Groups 9–10)

Opcode	Symbol	RTL Micro-Operations	Grp
0x11	AND	D17 T₂: $S_1 \leftarrow \text{SREG2}$ D17 T₃: $\text{DSTREG} \leftarrow \text{SREG1 AND SREG2}, \text{SC} \leftarrow 0.$	9
0x12	ORR	D18 T₂: $S_1 \leftarrow \text{SREG2}$ D18 T₃: $\text{DSTREG} \leftarrow \text{SREG1 OR SREG2}, \text{SC} \leftarrow 0.$	9
0x13	XOR	D19 T₂: $S_1 \leftarrow \text{SREG2}$ D19 T₃: $\text{DSTREG} \leftarrow \text{SREG1 XOR SREG2}, \text{SC} \leftarrow 0.$	9
0x14	NAND	D20 T₂: $S_1 \leftarrow \text{SREG2}$ D20 T₃: $\text{DSTREG} \leftarrow \text{SREG1 NAND SREG2}, \text{SC} \leftarrow 0.$	9
0x15	ADD	D21 T₂: $S_1 \leftarrow \text{SREG2}$ D21 T₃: $\text{DSTREG} \leftarrow \text{SREG1} + \text{SREG2}, \text{SC} \leftarrow 0.$	9
0x16	ADC	D22 T₂: $S_1 \leftarrow \text{SREG2}$ D22 T₃: $\text{DSTREG} \leftarrow \text{SREG1} + \text{SREG2} + \text{Carry}, \text{SC} \leftarrow 0.$	9
0x17	SUB	D23 T₂: $S_1 \leftarrow \text{SREG2}$ D23 T₃: $\text{DSTREG} \leftarrow \text{SREG1} - \text{SREG2}, \text{SC} \leftarrow 0.$	9
0x18	MOV	D24 T₂: $\text{DSTREG} \leftarrow \text{SREG1}, \text{SC} \leftarrow 0.$	10

Group 9: Two-Operand ALU Operations Opcodes 0x11–0x17 perform all of our binary ALU instructions (AND, OR, XOR, NAND, ADD, ADC, SUB) over two microcycles. In the first cycle ($D17T_2$ – $D23T_2$), the second source register (SREG2) is loaded into

the ALU's S_1 input. In the second cycle ($D17T_3$ – $D23T_3$), the ALU computes the specified logical or arithmetic operation with $SREG1$, writes the result into $DSTREG$, updates the flags, and then clears the sequence counter (SC).

Group 10: Register-to-Register Move Opcode $0x18$ (MOV) also routes $SREG1$ through the ALU, updates flags and then writes the result into $DSTREG$ in a single micro-cycle ($D24T_2$) before clearing SC.

These five tables together cover the complete RTL micro-operation sequences for all of our CPU's addressing and non-addressing instructions.

3 RESULTS [15 points]

To verify the performance of our instruction, we built some simulation files and checked the results of the instruction implementation. In addition to that, we managed to pass all test cases in the homework.

3.1 Results from Simulations We Built

```

Output Values:
T: 1
Address Register File: PC: 0, AR: 0, SP: ff
Instruction Register :    0
Register File Registers: R1: 0, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 0, S2: 0, S3: 0, S4: 0
ALU Flags: Z: 0, N: 0, C: 0, O: 0
ALU Result: ALUOut: 0

Output Values:
T: 2
Address Register File: PC: 1, AR: 0, SP: ff
Instruction Register :   18
Register File Registers: R1: 0, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 0, S2: 0, S3: 0, S4: 0
ALU Flags: Z: 0, N: 0, C: 0, O: 0
ALU Result: ALUOut: 0

Output Values:
T: 4
Address Register File: PC: 2, AR: 0, SP: ff
Instruction Register :   18
Register File Registers: R1: 0, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 0, S2: 0, S3: 0, S4: 0
ALU Flags: Z: 0, N: 0, C: 0, O: 0
ALU Result: ALUOut: 0

Output Values:
T: 1
Address Register File: PC: 18, AR: 0, SP: ff
Instruction Register :   18
Register File Registers: R1: 0, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 0, S2: 0, S3: 0, S4: 0
ALU Flags: Z: 0, N: 0, C: 0, O: 0
ALU Result: ALUOut: 0

```

Figure 2: Execution of BRA

```

Output Values:
T: 1
Address Register File: PC: 0, AR: 0, SP: ff
Instruction Register :    0
Register File Registers: R1: -1, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 0, S2: 0, S3: 0, S4: 0
ALU Flags: Z: 0, N: 0, C: 0, O: 0
ALU Result: ALUOut: 0

```

```

Output Values:
T: 2
Address Register File: PC: 1, AR: 0, SP: ff
Instruction Register :   40
Register File Registers: R1: -1, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 0, S2: 0, S3: 0, S4: 0
ALU Flags: Z: 0, N: 0, C: 0, O: 0
ALU Result: ALUOut: 0

```

```

Output Values:
T: 4
Address Register File: PC: 2, AR: 0, SP: ff
Instruction Register :  2640
Register File Registers: R1: -1, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 0, S2: 0, S3: 0, S4: 0
ALU Flags: Z: 0, N: 0, C: 0, O: 0
ALU Result: ALUOut: -1

```

Figure 3: Execution of INC

```

Output Values:
T: 8
Address Register File: PC: 2, AR: 0, SP: ff
Instruction Register :  2640
Register File Registers: R1: -1, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 0, S2: 0, S3: 0, S4: 0
ALU Flags: Z: 0, N: 0, C: 1, O: 0
ALU Result: ALUOut: -1

```

```

Output Values:
T: 16
Address Register File: PC: 2, AR: 0, SP: ff
Instruction Register :  2640
Register File Registers: R1: 0, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 0, S2: 0, S3: 0, S4: 0
ALU Flags: Z: 0, N: 0, C: 1, O: 0
ALU Result: ALUOut: 0

```

```

Output Values:
T: 1
Address Register File: PC: 2, AR: 0, SP: ff
Instruction Register :  2640
Register File Registers: R1: 0, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 0, S2: 0, S3: 0, S4: 0
ALU Flags: Z: 1, N: 0, C: 0, O: 0
ALU Result: ALUOut: 0

```

Figure 4: Execution of INC

```

Output Values:
T: 1
Address Register File: PC: 0, AR: 0, SP: ff
Instruction Register :    0
Register File Registers: R1: -1, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 0, S2: 0, S3: 0, S4: 0
ALU Flags: Z: 0, N: 0, C: 0, O: 0
ALU Result: ALUOut: 0

Output Values:
T: 2
Address Register File: PC: 1, AR: 0, SP: ff
Instruction Register :    c0
Register File Registers: R1: -1, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 0, S2: 0, S3: 0, S4: 0
ALU Flags: Z: 0, N: 0, C: 0, O: 0
ALU Result: ALUOut: 0

Output Values:
T: 4
Address Register File: PC: 2, AR: 0, SP: ff
Instruction Register :   2ec0
Register File Registers: R1: -1, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 0, S2: 0, S3: 0, S4: 0
ALU Flags: Z: 0, N: 0, C: 0, O: 0
ALU Result: ALUOut: -2

Output Values:
T: 1
Address Register File: PC: 2, AR: 0, SP: ff
Instruction Register :   2ec0
Register File Registers: R1: -1, R2: -2, R3: 0, R4: 0
Register File Scratch Registers: S1: 0, S2: 0, S3: 0, S4: 0
ALU Flags: Z: 0, N: 1, C: 1, O: 0
ALU Result: ALUOut: -2

```

Figure 5: Execution of LSL

```

Output Values:
T: 0
Address Register File: PC: 0, AR: 0, SP: ff
Instruction Register :    0
Data Register :   12567890
Register File Registers: R1: ffffffff, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 0, S2: 0, S3: 0, S4: 0
ALU Flags: Z: 0, N: 0, C: 0, O: 0
Memory: M[11]: 0, M[12]: 0, M[13]: 0, M[14]: 0
ALU Result: ALUOut: 0

Output Values:
T: 1
Address Register File: PC: 0, AR: 0, SP: ff
Instruction Register :    0
Data Register :   12567890
Register File Registers: R1: ffffffff, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 0, S2: 0, S3: 0, S4: 0
ALU Flags: Z: 0, N: 0, C: 0, O: 0
Memory: M[11]: 0, M[12]: 0, M[13]: 0, M[14]: 0
ALU Result: ALUOut: 0

Output Values:
T: 2
Address Register File: PC: 1, AR: 0, SP: ff
Instruction Register :    0
Data Register :   12567890
Register File Registers: R1: ffffffff, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 0, S2: 0, S3: 0, S4: 0
ALU Flags: Z: 0, N: 0, C: 0, O: 0
Memory: M[11]: 0, M[12]: 0, M[13]: 0, M[14]: 0
ALU Result: ALUOut: 0

```

Figure 6: Execution of LDARH

```

Output Values:
T: 4
Address Register File: PC: 2, AR: 0, SP: ff
Instruction Register : 7200
Data Register : 12567890
Register File Registers: R1: ffffffff, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 0, S2: 0, S3: 0, S4: 0
ALU Flags: Z: 0, N: 0, C: 0, O: 0
Memory: M[11]: 0, M[12]: 0, M[13]: 0, M[14]: 0
ALU Result: ALUOut: 0

Output Values:
T: 8
Address Register File: PC: 2, AR: 1, SP: ff
Instruction Register : 7200
Data Register : 0
Register File Registers: R1: ffffffff, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 0, S2: 0, S3: 0, S4: 0
ALU Flags: Z: 0, N: 0, C: 0, O: 0
Memory: M[11]: 0, M[12]: 0, M[13]: 0, M[14]: 0
ALU Result: ALUOut: 0

Output Values:
T: 16
Address Register File: PC: 2, AR: 2, SP: ff
Instruction Register : 7200
Data Register : 72
Register File Registers: R1: ffffffff, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 0, S2: 0, S3: 0, S4: 0
ALU Flags: Z: 0, N: 0, C: 0, O: 0
Memory: M[11]: 0, M[12]: 0, M[13]: 0, M[14]: 0
ALU Result: ALUOut: 0

```

Figure 7: Execution of LDARH

```

Output Values:
T: 32
Address Register File: PC: 2, AR: 3, SP: ff
Instruction Register : 7200
Data Register : 721e
Register File Registers: R1: ffffffff, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 0, S2: 0, S3: 0, S4: 0
ALU Flags: Z: 0, N: 0, C: 0, O: 0
Memory: M[11]: 0, M[12]: 0, M[13]: 0, M[14]: 0
ALU Result: ALUOut: 0

Output Values:
T: 64
Address Register File: PC: 2, AR: 3, SP: ff
Instruction Register : 7200
Data Register : 721elf
Register File Registers: R1: ffffffff, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 0, S2: 0, S3: 0, S4: 0
ALU Flags: Z: 0, N: 0, C: 0, O: 0
Memory: M[11]: 0, M[12]: 0, M[13]: 0, M[14]: 0
ALU Result: ALUOut: 0

Output Values:
T: 1
Address Register File: PC: 2, AR: 3, SP: ff
Instruction Register : 7200
Data Register : 721elf
Register File Registers: R1: 721elf, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 0, S2: 0, S3: 0, S4: 0
ALU Flags: Z: 0, N: 0, C: 0, O: 0
Memory: M[11]: 0, M[12]: 0, M[13]: 0, M[14]: 0
ALU Result: ALUOut: 0

```

Figure 8: Execution of LDARH

```

Output Values:
T: 1
Address Register File: PC: 0, AR: 0, SP: ff
Instruction Register : 0
Data Register : 0
Register File Registers: R1: -1, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 0, S2: 0, S3: 0, S4: 0
ALU Flags: Z: 0, N: 0, C: 0, O: 0
ALU Result: ALUOut: 0

Output Values:
T: 2
Address Register File: PC: 1, AR: 0, SP: ff
Instruction Register : 0
Data Register : 0
Register File Registers: R1: -1, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 0, S2: 0, S3: 0, S4: 0
ALU Flags: Z: 0, N: 0, C: 0, O: 0
ALU Result: ALUOut: 0

Output Values:
T: 4
Address Register File: PC: 2, AR: 0, SP: ff
Instruction Register : 8800
Data Register : 0
Register File Registers: R1: -1, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 0, S2: 0, S3: 0, S4: 0
ALU Flags: Z: 0, N: 0, C: 0, O: 0
ALU Result: ALUOut: 0

```

Figure 9: Execution of LDDRH

```

T: 8
Address Register File: PC: 2, AR: 1, SP: ff
Instruction Register : 8800
Data Register : 0
Register File Registers: R1: -1, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 0, S2: 0, S3: 0, S4: 0
ALU Flags: Z: 0, N: 0, C: 0, O: 0
ALU Result: ALUOut: 0

Output Values:
T: 16
Address Register File: PC: 2, AR: 2, SP: ff
Instruction Register : 8800
Data Register : 88
Register File Registers: R1: -1, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 0, S2: 0, S3: 0, S4: 0
ALU Flags: Z: 0, N: 0, C: 0, O: 0
ALU Result: ALUOut: 0

Output Values:
T: 32
Address Register File: PC: 2, AR: 3, SP: ff
Instruction Register : 8800
Data Register : 881e
Register File Registers: R1: -1, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 0, S2: 0, S3: 0, S4: 0
ALU Flags: Z: 0, N: 0, C: 0, O: 0
ALU Result: ALUOut: 0

Output Values:
T: 1
Address Register File: PC: 2, AR: 3, SP: ff
Instruction Register : 8800
Data Register : 881elf
Register File Registers: R1: -1, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 0, S2: 0, S3: 0, S4: 0
ALU Flags: Z: 0, N: 0, C: 0, O: 0
ALU Result: ALUOut: 0

```

Figure 10: Execution of LDDRH

```

T: 0
Address Register File: PC: 0, AR: 1, SP: ff
Instruction Register : 0
Data Register : 12567890
Register File Registers: R1: ffffffff, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 0, S2: 0, S3: 0, S4: 0
ALU Flags: Z: 0, N: 0, C: 0, O: 0
Memory: M[11]: 0, M[12]: 0, M[13]: 0, M[14]: 0
ALU Result: ALUOut: 0

```

Output Values:

```

T: 1
Address Register File: PC: 0, AR: 1, SP: ff
Instruction Register : 0
Data Register : 12567890
Register File Registers: R1: ffffffff, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 0, S2: 0, S3: 0, S4: 0
ALU Flags: Z: 0, N: 0, C: 0, O: 0
Memory: M[11]: 0, M[12]: 0, M[13]: 0, M[14]: 0
ALU Result: ALUOut: 0

```

Output Values:

```

T: 2
Address Register File: PC: 1, AR: 1, SP: ff
Instruction Register : 10
Data Register : 12567890
Register File Registers: R1: ffffffff, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 0, S2: 0, S3: 0, S4: 0
ALU Flags: Z: 0, N: 0, C: 0, O: 0
Memory: M[11]: 0, M[12]: 0, M[13]: 0, M[14]: 0
ALU Result: ALUOut: 0

```

Figure 11: Execution of STRIM

Output Values:

```

T: 4
Address Register File: PC: 2, AR: 1, SP: ff
Instruction Register : 9010
Data Register : 12567890
Register File Registers: R1: ffffffff, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 0, S2: 0, S3: 0, S4: 0
ALU Flags: Z: 0, N: 0, C: 0, O: 0
Memory: M[11]: 0, M[12]: 0, M[13]: 0, M[14]: 0
ALU Result: ALUOut: 0

```

Output Values:

```

T: 8
Address Register File: PC: 2, AR: 1, SP: ff
Instruction Register : 9010
Data Register : 12567890
Register File Registers: R1: ffffffff, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 16, S2: 0, S3: 0, S4: 0
ALU Flags: Z: 0, N: 0, C: 0, O: 0
Memory: M[11]: 0, M[12]: 0, M[13]: 0, M[14]: 0
ALU Result: ALUOut: 17

```

Output Values:

```

T: 16
Address Register File: PC: 2, AR: 11, SP: ff
Instruction Register : 9010
Data Register : 12567890
Register File Registers: R1: ffffffff, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 16, S2: 0, S3: 0, S4: 0
ALU Flags: Z: 0, N: 0, C: 0, O: 0
Memory: M[11]: 0, M[12]: 0, M[13]: 0, M[14]: 0
ALU Result: ALUOut: -1

```

Figure 12: Execution of STRIM

```

Output Values:
T: 32
Address Register File: PC: 2, AR: 12, SP: ff
Instruction Register : 9010
Data Register : 12567890
Register File Registers: R1: ffffffff, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 16, S2: 0, S3: 0, S4: 0
ALU Flags: Z: 0, N: 0, C: 0, O: 0
Memory: M[11]: ff, M[12]: 0, M[13]: 0, M[14]: 0
ALU Result: ALUOut: -1

Output Values:
T: 64
Address Register File: PC: 2, AR: 13, SP: ff
Instruction Register : 9010
Data Register : 12567890
Register File Registers: R1: ffffffff, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 16, S2: 0, S3: 0, S4: 0
ALU Flags: Z: 0, N: 0, C: 0, O: 0
Memory: M[11]: ff, M[12]: ff, M[13]: 0, M[14]: 0
ALU Result: ALUOut: -1

Output Values:
T: 128
Address Register File: PC: 2, AR: 14, SP: ff
Instruction Register : 9010
Data Register : 12567890
Register File Registers: R1: ffffffff, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 16, S2: 0, S3: 0, S4: 0
ALU Flags: Z: 0, N: 0, C: 0, O: 0
Memory: M[11]: ff, M[12]: ff, M[13]: ff, M[14]: 0
ALU Result: ALUOut: -1

```

Figure 13: Execution of STRIM

```

Output Values:
T: 128
Address Register File: PC: 2, AR: 14, SP: ff
Instruction Register : 9010
Data Register : 12567890
Register File Registers: R1: ffffffff, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 16, S2: 0, S3: 0, S4: 0
ALU Flags: Z: 0, N: 0, C: 0, O: 0
Memory: M[11]: ff, M[12]: ff, M[13]: ff, M[14]: 0
ALU Result: ALUOut: -1

Output Values:
T: 1
Address Register File: PC: 2, AR: 14, SP: ff
Instruction Register : 9010
Data Register : 12567890
Register File Registers: R1: ffffffff, R2: 0, R3: 0, R4: 0
Register File Scratch Registers: S1: 16, S2: 0, S3: 0, S4: 0
ALU Flags: Z: 0, N: 0, C: 0, O: 0
Memory: M[11]: ff, M[12]: ff, M[13]: ff, M[14]: ff
ALU Result: ALUOut: -1

```

Figure 14: Execution of STRIM

3.2 Results from Instructor-Provided Simulations

In this project, we were able to pass all the test cases provided in the simulation file. The results are shown in the following figures.

```
-----
CPUSystem Simulation Started
[PASS] Test No: 1, Component: R2, Actual Value: 0x77777777, Expected Value: 0x77777777
[PASS] Test No: 1, Component: R2, Actual Value: 0x00000000, Expected Value: 0x00000000
[PASS] Test No: 2, Component: R1, Actual Value: 0x00000001, Expected Value: 0x00000001
[PASS] Test No: 2, Component: Z, Actual Value: 0x00000000, Expected Value: 0x00000000
[PASS] Test No: 3, Component: R1, Actual Value: 0x00000001, Expected Value: 0x00000001
[PASS] Test No: 3, Component: R2, Actual Value: 0x00000000, Expected Value: 0x00000000
[PASS] Test No: 3, Component: Z, Actual Value: 0x00000001, Expected Value: 0x00000001
[PASS] Test No: 4, Component: DR0ut, Actual Value: 0x00000600, Expected Value: 0x00000600
[PASS] Test No: 4, Component: R3, Actual Value: 0x00000600, Expected Value: 0x00000600
[PASS] Test No: 4, Component: AR, Actual Value: 0x00000001, Expected Value: 0x00000001
[PASS] Test No: 5, Component: DR0ut, Actual Value: 0x06000a08, Expected Value: 0x06000a08
[PASS] Test No: 5, Component: R3, Actual Value: 0x06000a08, Expected Value: 0x06000a08
[PASS] Test No: 5, Component: AR, Actual Value: 0x00000003, Expected Value: 0x00000003
[PASS] Test No: 6, Component: R3, Actual Value: 0x06000a08, Expected Value: 0x06000a08
[PASS] Test No: 6, Component: AR, Actual Value: 0x0000000b, Expected Value: 0x0000000b
[PASS] Test No: 6, Component: MEM[AR], Actual Value: 0x00000006, Expected Value: 0x00000006
[PASS] Test No: 6, Component: MEM[AR], Actual Value: 0x00000000, Expected Value: 0x00000000
[PASS] Test No: 6, Component: MEM[AR], Actual Value: 0x0000000a, Expected Value: 0x0000000a
[PASS] Test No: 6, Component: MEM[AR], Actual Value: 0x00000008, Expected Value: 0x00000008
[PASS] Test No: 7, Component: PC, Actual Value: 0x00000032, Expected Value: 0x00000032
[PASS] Test No: 7, Component: SP, Actual Value: 0x000000fd, Expected Value: 0x000000fd
[PASS] Test No: 7, Component: MEM[SP], Actual Value: 0x000000aa, Expected Value: 0x000000aa
[PASS] Test No: 7, Component: MEM[SP], Actual Value: 0x000000bb, Expected Value: 0x000000bb
[PASS] Test No: 8, Component: R1, Actual Value: 0x000000fe, Expected Value: 0x000000fe
[PASS] Test No: 8, Component: AR, Actual Value: 0x00000012, Expected Value: 0x00000012
CPUSystem Simulation Finished
0 Test Failed
25 Test Passed
```

Figure 15: The result of running CPUSystemSimulation.v

```
-----
CPUSystem Simulation Started
[PASS] Test No: 70, Component: IROut, Actual Value: 0x00000052, Expected Value: 0x00000052
[PASS] Test No: 70, Component: PC, Actual Value: 0x00000052, Expected Value: 0x00000052
[PASS] Test No: 70, Component: SP, Actual Value: 0x000000ff, Expected Value: 0x000000ff
[PASS] Test No: 70, Component: AR, Actual Value: 0x00000057, Expected Value: 0x00000057
[PASS] Test No: 70, Component: MEM[ADDRESS], Actual Value: 0x00000000, Expected Value: 0x00000000
[PASS] Test No: 70, Component: MEM[ADDRESS], Actual Value: 0x00000000, Expected Value: 0x00000000
[PASS] Test No: 70, Component: MEM[ADDRESS], Actual Value: 0x00000000, Expected Value: 0x00000000
[PASS] Test No: 70, Component: MEM[ADDRESS], Actual Value: 0x00000018, Expected Value: 0x00000018
[PASS] Test No: 70, Component: R1, Actual Value: 0x00000001, Expected Value: 0x00000001
[PASS] Test No: 70, Component: R2, Actual Value: 0x00000018, Expected Value: 0x00000018
[PASS] Test No: 70, Component: R3, Actual Value: 0x00000006, Expected Value: 0x00000006
[PASS] Test No: 70, Component: R4, Actual Value: 0x00000000, Expected Value: 0x00000000
[PASS] Test No: 70, Component: MEM[SP], Actual Value: 0x00000000, Expected Value: 0x00000000
[PASS] Test No: 70, Component: MEM[SP], Actual Value: 0x00000040, Expected Value: 0x00000040
[PASS] Test No: 70, Component: MEM[SP], Actual Value: 0x00000000, Expected Value: 0x00000000
[PASS] Test No: 70, Component: MEM[SP], Actual Value: 0x00000044, Expected Value: 0x00000044
[PASS] Test No: 70, Component: MEM[SP], Actual Value: 0x00000000, Expected Value: 0x00000000
[PASS] Test No: 70, Component: MEM[SP], Actual Value: 0x00000040, Expected Value: 0x00000040
[PASS] Test No: 70, Component: MEM[SP], Actual Value: 0x00000000, Expected Value: 0x00000000
[PASS] Test No: 70, Component: MEM[SP], Actual Value: 0x00000044, Expected Value: 0x00000044
[PASS] Test No: 70, Component: MEM[SP], Actual Value: 0x00000000, Expected Value: 0x00000000
[PASS] Test No: 70, Component: MEM[SP], Actual Value: 0x00000040, Expected Value: 0x00000040
[PASS] Test No: 70, Component: MEM[SP], Actual Value: 0x00000000, Expected Value: 0x00000000
[PASS] Test No: 70, Component: MEM[SP], Actual Value: 0x00000044, Expected Value: 0x00000044
[PASS] Test No: 70, Component: MEM[SP], Actual Value: 0x00000000, Expected Value: 0x00000000
[PASS] Test No: 70, Component: MEM[SP], Actual Value: 0x0000002e, Expected Value: 0x0000002e
CPUSystem Simulation Finished
0 Test Failed
26 Test Passed
```

Figure 16: The result of running CPUSystemSimulation_Factorial.v

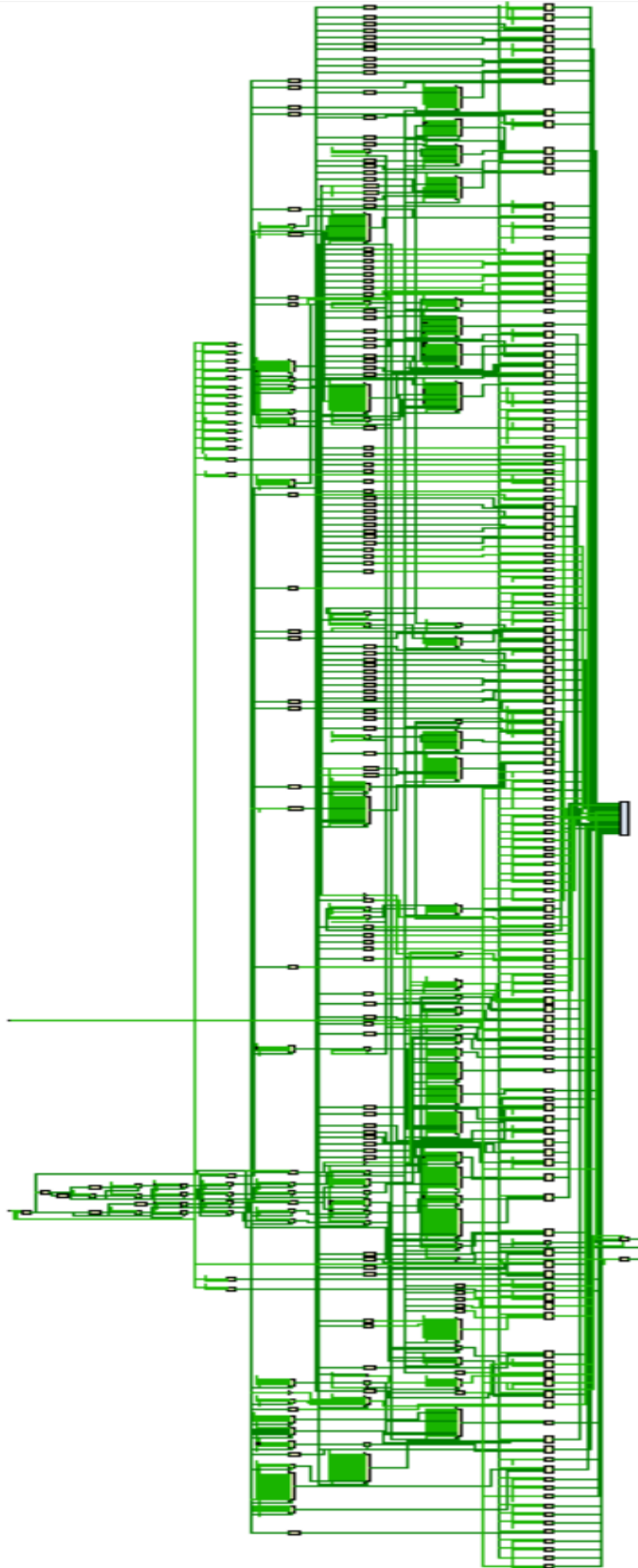


Figure 17: Schematic of CPU System

3.3 Results from Simple Example

We were asked to modify the value in memory to implement a simple example. In the end, we obtained the expected result: R1 decreased from 6 to zero as expected. Additionally, we observed that the memory contained zero values from location x'B0 to x'100, which explains why the sum in R2 was zero. Moreover, as expected, R3 has the value x'B0 at the end of the simulation. Following figure shows the signal behavior in Simple Example.

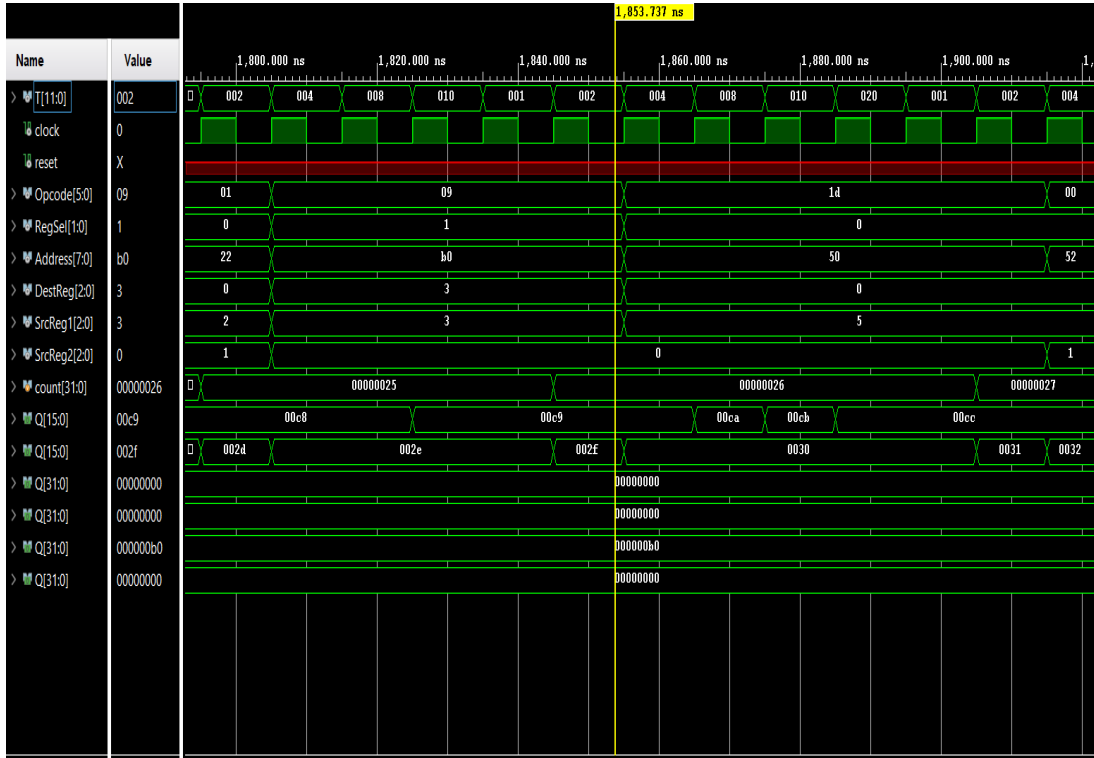


Figure 18: Signal's Behavior in Simple Example

4 DISCUSSION

During implementation, we classified the 36 instructions into logical groups to reveal common micro-operation patterns and simplify our `case` implementation in Verilog. The groups include control flow operations, stack operations, subroutines, immediate loads, memory transfers, single-operand shifts/NOT, two-operand ALU ops, etc. This grouping not only boosted readability and manageability, but also guided our RTL design and helped us spot bugs during code debugging process.

We practiced precise control-signal timing using our `disable_reg()` task, invoked at the start of every microcycle. By first clearing all register enables and control signals, we prevented “leftover” enables from one instruction cycle affecting the next, thereby

eliminating numerous overwriting bugs.

Verification is made using custom simulation files that exercised each instruction in isolation and the tests provided by the assignment. Whenever an output deviated from the expected RTL behavior spotted by the tests and simulations, we paused the run, examined the waveform of every control signal, and manually inspected memory contents after each micro-operation. This comprehensive debugging sharpened our skills in tracing signal interactions and reinforced our understanding of the sequence counter, ALU flags, and register file interdependencies. Moreover, since these test files do not check the whole instruction set we implemented our test files and checked them. This helped us fix a lot of issues that we would not have noticed if we had not implemented these instructions. We also improved our Project 1 ALU subsystem by adding the full suite of shift and logical functions and ensuring that arithmetic instructions route their results through the ALU in the final cycle to update the condition flags. Integrating these enhancements taught us valuable lessons in modular design. We provided clear and descriptive comments at every microcycle throughout our Verilog code implementation. These annotations document the RTL intent, flags's behavior, and register enables, making future adjustments or extensions easier and reduce the risk of introducing regressions.

5 CONCLUSION

By combining thoughtful grouping, rigorous control-signal discipline, manual memory checks, and targeted ALU improvements, we successfully implemented the required hardwired control unit for our CPU system. Along the way, we honed our hardwired design skills and learned the principles of control-unit architecture. We also created new simulation sources and leveraged Vivado's debugging features to validate and troubleshoot each instruction. We have faced challenges in understanding task/procedure semantics, and precise timing. These are motivated us to deepen our HDL expertise. In the end, our perseverance paid off: we passed all the tests, verified instruction behaviour and we have built and tested a basic computer.